

NAME - SARVESH THAKUR

ROLL NO - GF202567531

PROJECT - CAR PRICE PREDICTION

GROUP - 5

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
In [2]: df = pd.read_csv(r"C:\Users\ASUS\Documents\car data (1).csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

|   | Car_Name | Year | Selling_Price | Present_Price | Kms_Driven | Fuel_Type | Seller_Type | Trans |
|---|----------|------|---------------|---------------|------------|-----------|-------------|-------|
| 0 | ritz     | 2014 | 3.35          | 5.59          | 27000      | Petrol    | Dealer      |       |
| 1 | sx4      | 2013 | 4.75          | 9.54          | 43000      | Diesel    | Dealer      |       |
| 2 | ciaz     | 2017 | 7.25          | 9.85          | 6900       | Petrol    | Dealer      |       |
| 3 | wagon r  | 2011 | 2.85          | 4.15          | 5200       | Petrol    | Dealer      |       |
| 4 | swift    | 2014 | 4.60          | 6.87          | 42450      | Diesel    | Dealer      |       |

```
In [4]: df.shape
```

```
Out[4]: (301, 9)
```

```
In [5]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Car_Name        301 non-null   object
 1   Year            301 non-null   int64
 2   Selling_Price   301 non-null   float64
 3   Present_Price   301 non-null   float64
 4   Kms_Driven      301 non-null   int64
 5   Fuel_Type       301 non-null   object
 6   Seller_Type     301 non-null   object
 7   Transmission    301 non-null   object
 8   Owner           301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB

```

In [6]: `df.describe()`

Out[6]:

|              | Year        | Selling_Price | Present_Price | Kms_Driven    | Owner      |
|--------------|-------------|---------------|---------------|---------------|------------|
| <b>count</b> | 301.000000  | 301.000000    | 301.000000    | 301.000000    | 301.000000 |
| <b>mean</b>  | 2013.627907 | 4.661296      | 7.628472      | 36947.205980  | 0.043189   |
| <b>std</b>   | 2.891554    | 5.082812      | 8.644115      | 38886.883882  | 0.247915   |
| <b>min</b>   | 2003.000000 | 0.100000      | 0.320000      | 500.000000    | 0.000000   |
| <b>25%</b>   | 2012.000000 | 0.900000      | 1.200000      | 15000.000000  | 0.000000   |
| <b>50%</b>   | 2014.000000 | 3.600000      | 6.400000      | 32000.000000  | 0.000000   |
| <b>75%</b>   | 2016.000000 | 6.000000      | 9.900000      | 48767.000000  | 0.000000   |
| <b>max</b>   | 2018.000000 | 35.000000     | 92.600000     | 500000.000000 | 3.000000   |

In [7]: `df.isnull().sum()`

Out[7]:

```

Car_Name      0
Year          0
Selling_Price 0
Present_Price 0
Kms_Driven    0
Fuel_Type     0
Seller_Type    0
Transmission  0
Owner         0
dtype: int64

```

In [8]: `df.duplicated().sum()`

Out[8]: `np.int64(2)`

In [9]: `df.drop_duplicates(inplace=True)`

```
In [10]: df.reset_index(drop=True, inplace=True)
```

```
In [11]: df['Current_Year'] = 2024
df['Car_Age'] = df['Current_Year'] - df['Year']
df.head()
```

```
Out[11]:
```

|   | Car_Name | Year | Selling_Price | Present_Price | Kms_Driven | Fuel_Type | Seller_Type | Trans |
|---|----------|------|---------------|---------------|------------|-----------|-------------|-------|
| 0 | ritz     | 2014 | 3.35          | 5.59          | 27000      | Petrol    | Dealer      |       |
| 1 | sx4      | 2013 | 4.75          | 9.54          | 43000      | Diesel    | Dealer      |       |
| 2 | ciaz     | 2017 | 7.25          | 9.85          | 6900       | Petrol    | Dealer      |       |
| 3 | wagon r  | 2011 | 2.85          | 4.15          | 5200       | Petrol    | Dealer      |       |
| 4 | swift    | 2014 | 4.60          | 6.87          | 42450      | Diesel    | Dealer      |       |

```
In [12]: df.drop(columns=['Car_Name', 'Year', 'Current_Year'], inplace=True)
```

```
In [13]: df.head()
```

```
Out[13]:
```

|   | Selling_Price | Present_Price | Kms_Driven | Fuel_Type | Seller_Type | Transmission | Owner |
|---|---------------|---------------|------------|-----------|-------------|--------------|-------|
| 0 | 3.35          | 5.59          | 27000      | Petrol    | Dealer      | Manual       | 0     |
| 1 | 4.75          | 9.54          | 43000      | Diesel    | Dealer      | Manual       | 0     |
| 2 | 7.25          | 9.85          | 6900       | Petrol    | Dealer      | Manual       | 0     |
| 3 | 2.85          | 4.15          | 5200       | Petrol    | Dealer      | Manual       | 0     |
| 4 | 4.60          | 6.87          | 42450      | Diesel    | Dealer      | Manual       | 0     |

```
In [14]: df['Fuel_Type'].value_counts()
```

```
Out[14]: Fuel_Type
Petrol    239
Diesel    58
CNG        2
Name: count, dtype: int64
```

```
In [15]: df['Seller_Type'].value_counts()
```

```
Out[15]: Seller_Type
Dealer    193
Individual 106
Name: count, dtype: int64
```

```
In [16]: df['Transmission'].value_counts()
```

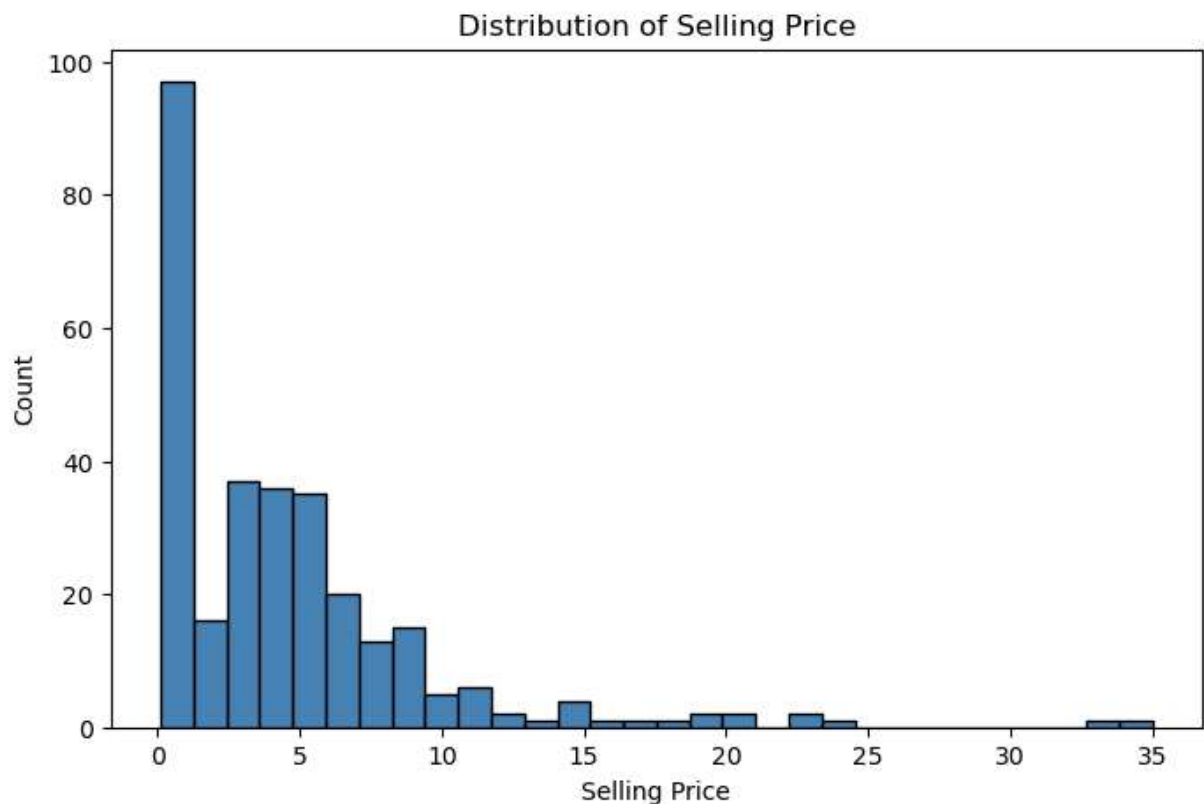
```
Out[16]: Transmission
Manual      260
Automatic   39
Name: count, dtype: int64
```

```
In [17]: le = LabelEncoder()
df['Fuel_Type'] = le.fit_transform(df['Fuel_Type'])
df['Seller_Type'] = le.fit_transform(df['Seller_Type'])
df['Transmission'] = le.fit_transform(df['Transmission'])
df.head()
```

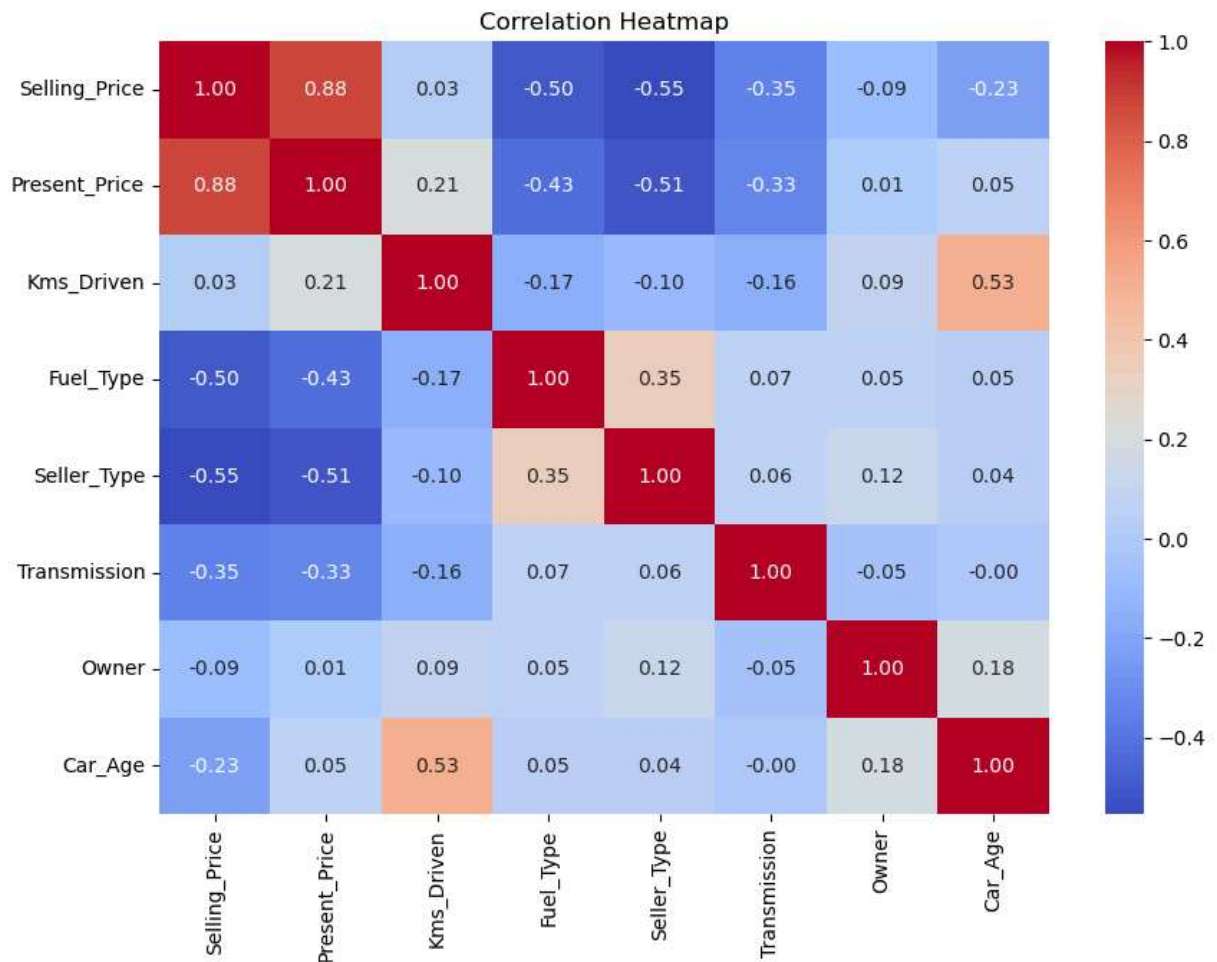
```
Out[17]:
```

|   | Selling_Price | Present_Price | Kms_Driven | Fuel_Type | Seller_Type | Transmission | Owner |
|---|---------------|---------------|------------|-----------|-------------|--------------|-------|
| 0 | 3.35          | 5.59          | 27000      | 2         | 0           | 1            | 0     |
| 1 | 4.75          | 9.54          | 43000      | 1         | 0           | 1            | 0     |
| 2 | 7.25          | 9.85          | 6900       | 2         | 0           | 1            | 0     |
| 3 | 2.85          | 4.15          | 5200       | 2         | 0           | 1            | 0     |
| 4 | 4.60          | 6.87          | 42450      | 1         | 0           | 1            | 0     |

```
In [18]: plt.figure(figsize=(8,5))
plt.hist(df['Selling_Price'], bins=30, color='steelblue', edgecolor='black')
plt.title('Distribution of Selling Price')
plt.xlabel('Selling Price')
plt.ylabel('Count')
plt.show()
```



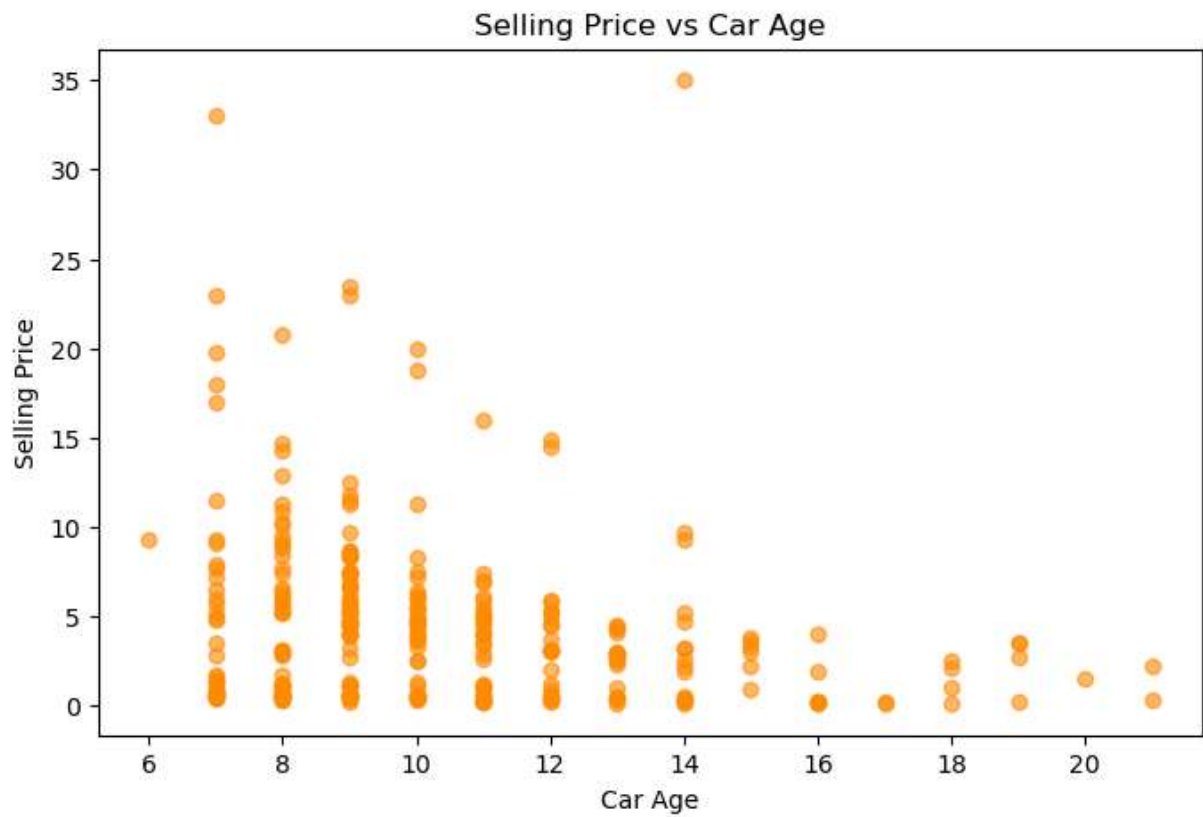
```
In [19]: plt.figure(figsize=(10,7))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



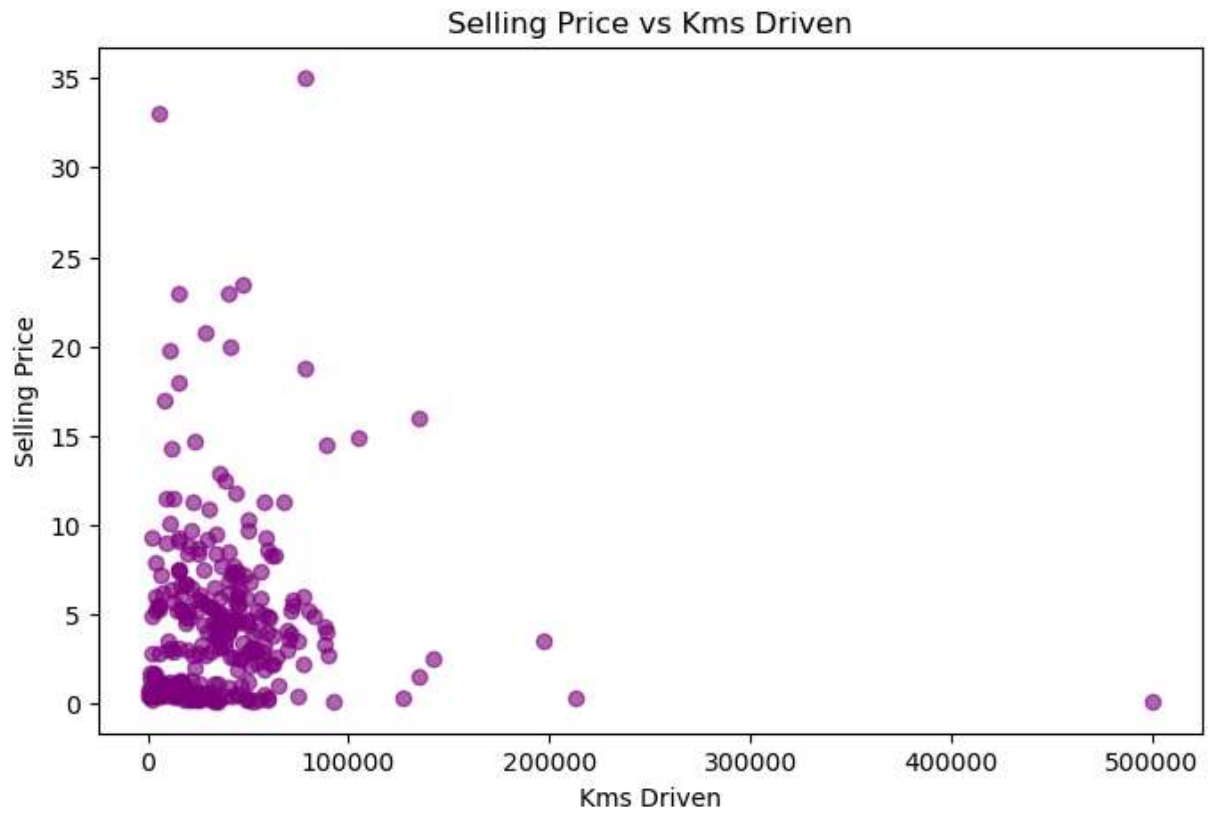
```
In [20]: plt.figure(figsize=(8,5))
plt.scatter(df['Present_Price'], df['Selling_Price'], color='teal', alpha=0.6)
plt.xlabel('Present Price')
plt.ylabel('Selling Price')
plt.title('Selling Price vs Present Price')
plt.show()
```



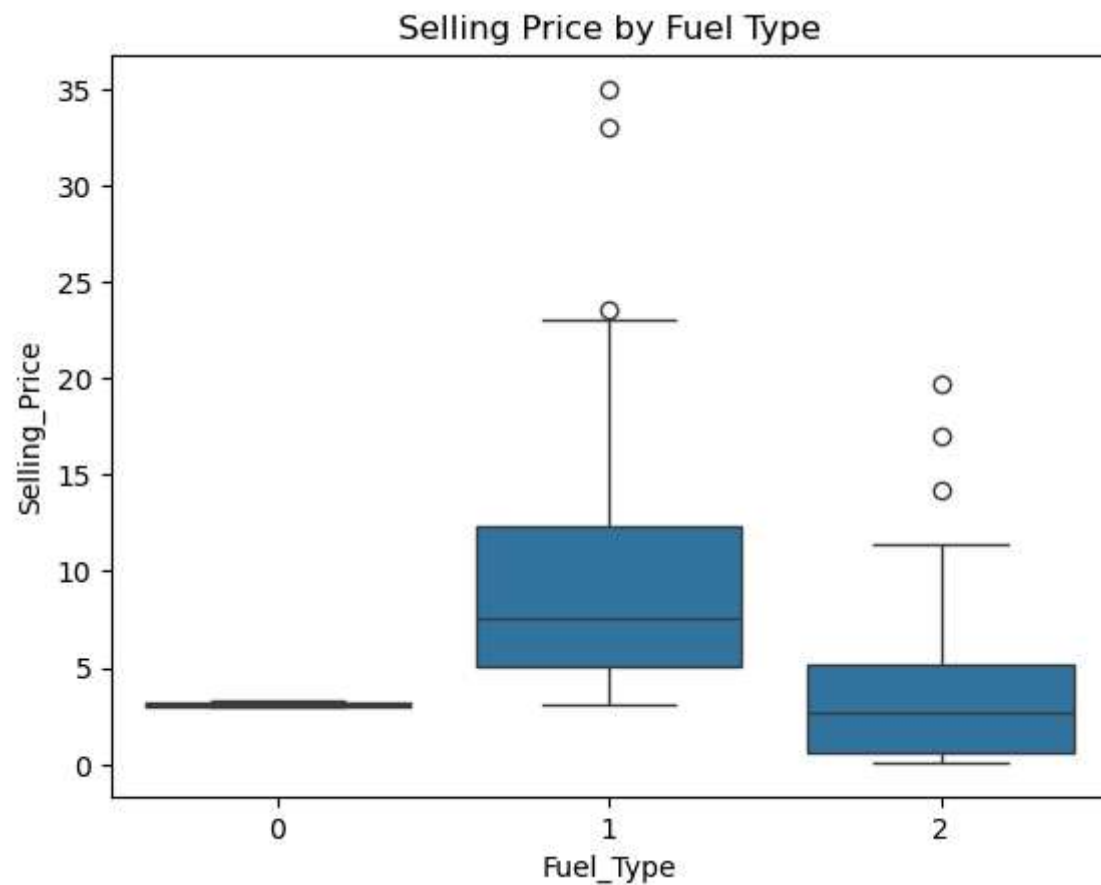
```
In [21]: plt.figure(figsize=(8,5))
plt.scatter(df['Car_Age'], df['Selling_Price'], color='darkorange', alpha=0.6)
plt.xlabel('Car Age')
plt.ylabel('Selling Price')
plt.title('Selling Price vs Car Age')
plt.show()
```



```
In [22]: plt.figure(figsize=(8,5))
plt.scatter(df['Kms_Driven'], df['Selling_Price'], color='purple', alpha=0.6)
plt.xlabel('Kms Driven')
plt.ylabel('Selling Price')
plt.title('Selling Price vs Kms Driven')
plt.show()
```

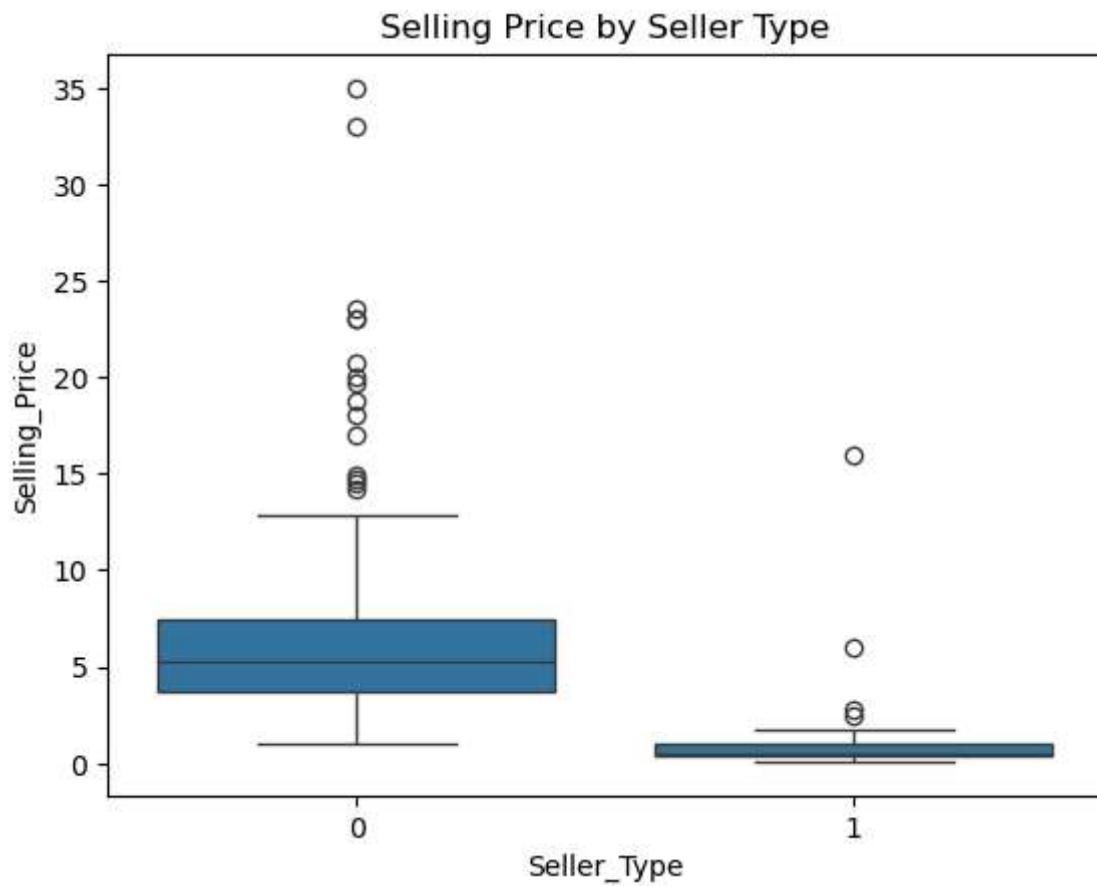


```
In [23]: sns.boxplot(x='Fuel_Type', y='Selling_Price', data=df)
plt.title('Selling Price by Fuel Type')
plt.show()
```





```
In [24]: sns.boxplot(x='Seller_Type', y='Selling_Price', data=df)
plt.title('Selling Price by Seller Type')
plt.show()
```



```
In [25]: sns.boxplot(x='Transmission', y='Selling_Price', data=df)
plt.title('Selling Price by Transmission')
plt.show()
```



```
In [26]: X = df.drop(columns=['Selling_Price'])
        y = df['Selling_Price']
```

```
In [27]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
        print(X_train.shape, X_test.shape)
```

```
(239, 7) (60, 7)
```

```
In [28]: lr = LinearRegression()
        lr.fit(X_train, y_train)
        y_pred_lr = lr.predict(X_test)
```

```
In [29]: print("Linear Regression")
        print("MAE :", mean_absolute_error(y_test, y_pred_lr))
        print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
        print("R2  :", r2_score(y_test, y_pred_lr))
```

```
Linear Regression
MAE : 1.5410718122837714
RMSE: 2.583241654748597
R2  : 0.7410829335731293
```

```
In [30]: dt = DecisionTreeRegressor(max_depth=8, random_state=42)
        dt.fit(X_train, y_train)
        y_pred_dt = dt.predict(X_test)
```

```
In [32]: print("Decision Tree")
        print("MAE :", mean_absolute_error(y_test, y_pred_dt))
```

```
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_dt)))
print("R2 :", r2_score(y_test, y_pred_dt))
```

Decision Tree

MAE : 1.314507341269841  
RMSE: 2.968419765800069  
R2 : 0.6581140629378662

```
In [33]: rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

```
In [34]: print("Random Forest")
print("MAE :", mean_absolute_error(y_test, y_pred_rf))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_rf)))
print("R2 :", r2_score(y_test, y_pred_rf))
```

Random Forest

MAE : 1.4000100000000001  
RMSE: 3.298996636857941  
R2 : 0.5777259891991424

```
In [35]: gb = GradientBoostingRegressor(n_estimators=200, learning_rate=0.1, random_state=42)
gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)
```

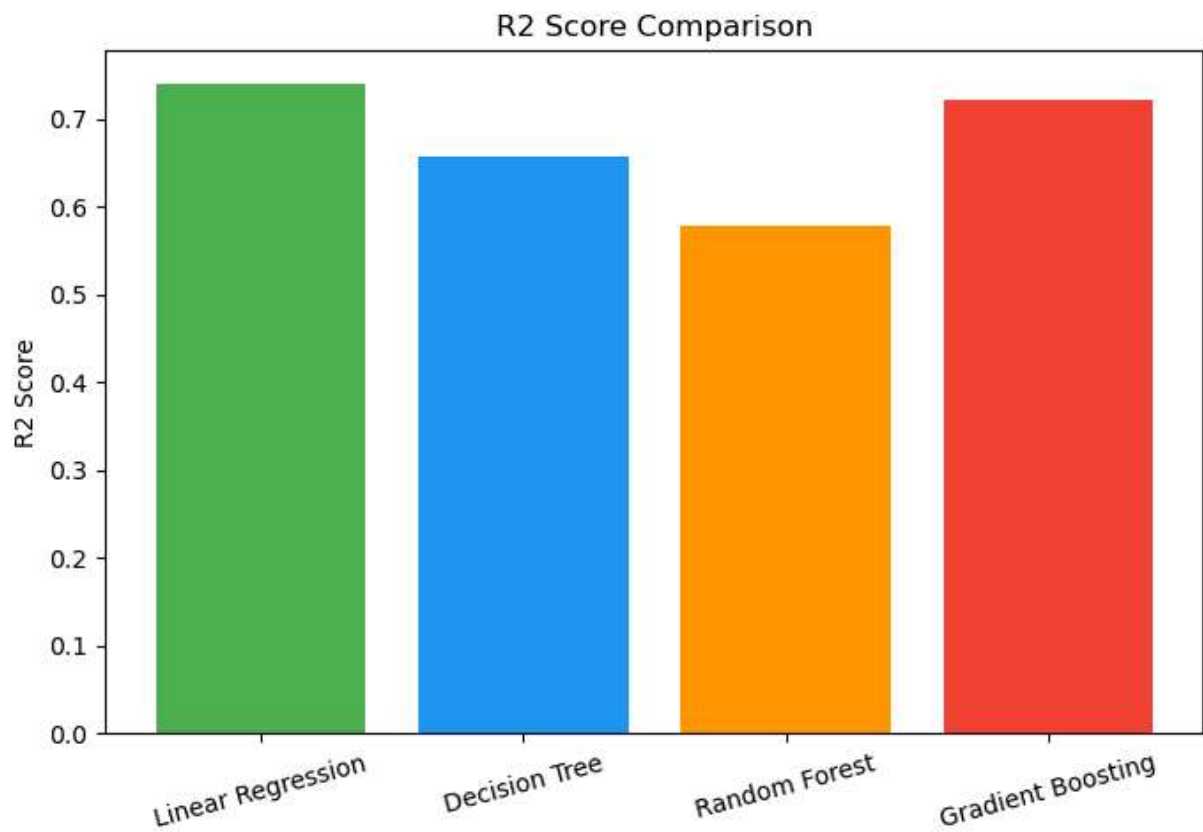
```
In [36]: print("Gradient Boosting")
print("MAE :", mean_absolute_error(y_test, y_pred_gb))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_gb)))
print("R2 :", r2_score(y_test, y_pred_gb))
```

Gradient Boosting

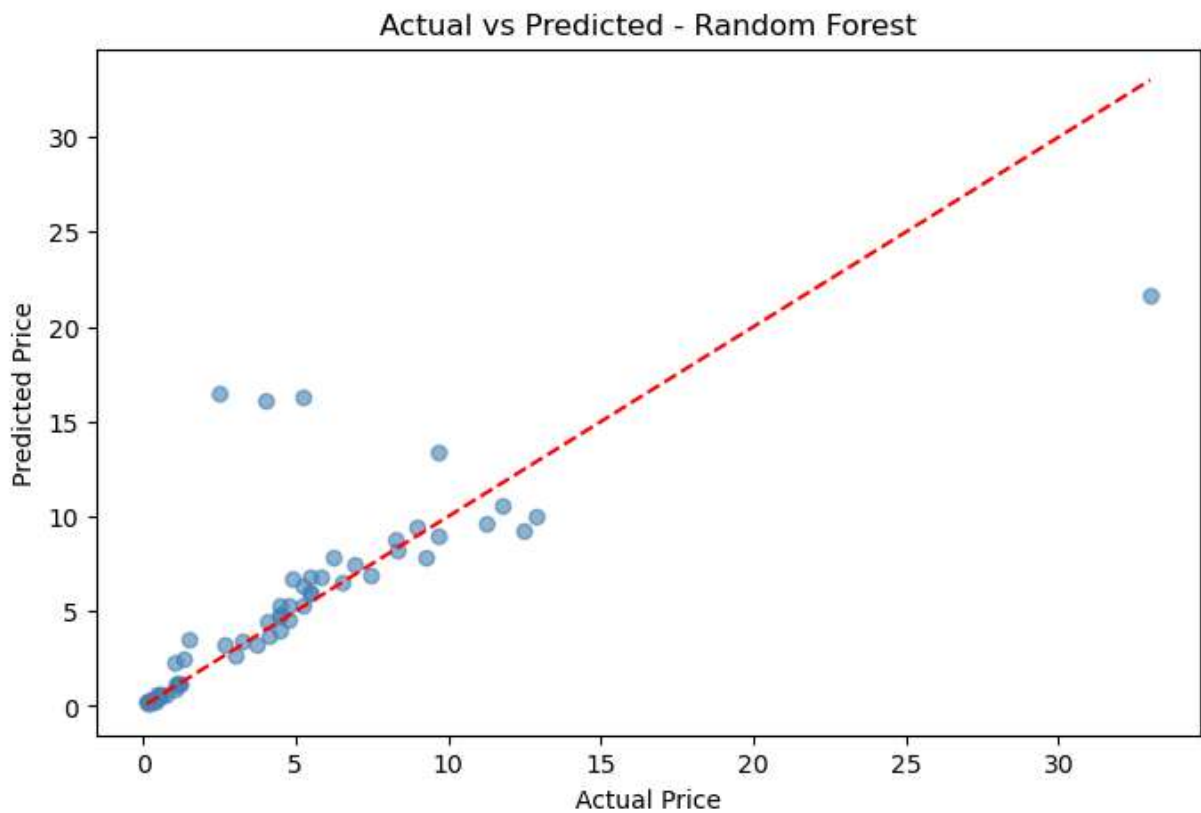
MAE : 1.2050249207898374  
RMSE: 2.6788387646191256  
R2 : 0.7215650442325969

```
In [37]: models = ['Linear Regression', 'Decision Tree', 'Random Forest', 'Gradient Boosting']
r2_scores = [
    r2_score(y_test, y_pred_lr),
    r2_score(y_test, y_pred_dt),
    r2_score(y_test, y_pred_rf),
    r2_score(y_test, y_pred_gb)
]
```

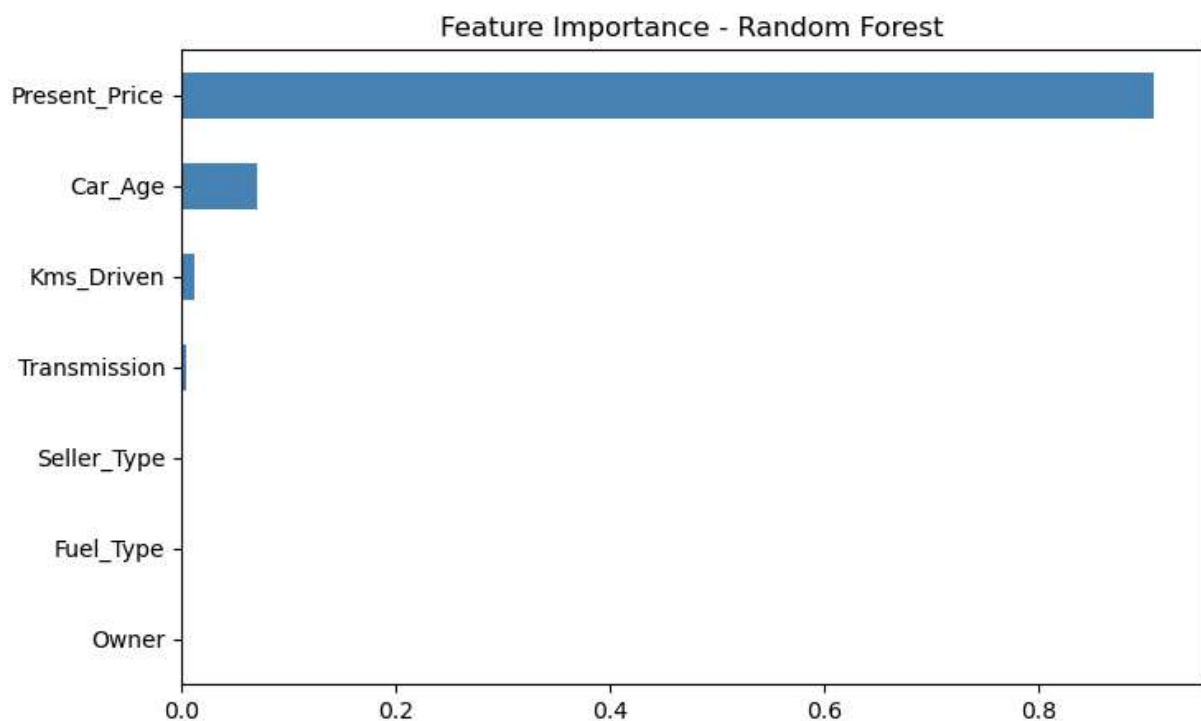
```
In [38]: plt.figure(figsize=(8,5))
plt.bar(models, r2_scores, color=['#4CAF50', '#2196F3', '#FF9800', '#F44336'])
plt.title('R2 Score Comparison')
plt.ylabel('R2 Score')
plt.xticks(rotation=15)
plt.show()
```



```
In [40]: plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred_rf, alpha=0.6, color='steelblue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted - Random Forest')
plt.show()
```



```
In [41]: feat_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values()
feat_imp.plot(kind='barh', color='steelblue', figsize=(8,5))
plt.title('Feature Importance - Random Forest')
plt.show()
```



```
In [42]: # Fuel_Type    -> CNG=0, Diesel=1, Petrol=2  
# Seller_Type  -> Dealer=0, Individual=1  
# Transmission -> Automatic=0, Manual=1
```

```
In [44]: new_car = pd.DataFrame({  
    'Present_Price': [6.5],  
    'Kms_Driven':    [40000],  
    'Fuel_Type':     [2],  
    'Seller_Type':   [0],  
    'Transmission': [1],  
    'Owner':         [0],  
    'Car_Age':       [5]  
})
```

```
In [46]: predicted_price = rf.predict(new_car)[0]  
print(f"Predicted Selling Price: {predicted_price:.2f} Lakhs")
```

Predicted Selling Price: 4.33 Lakhs